5.4 Гонка данных, состояние гонки и мьютексы в GoLang

Ключевые моменты

- **Гонка данных**: Возникает при одновременном доступе нескольких горутин к общей переменной, если хотя бы одна из них пишет.
- **Состояние гонки**: Некорректное поведение программы из-за отсутствия синхронизации при конкурентном доступе.
- **Мьютексы**: Обеспечивают эксклюзивный доступ к критической секции, предотвращая гонки данных.
- **Критическая секция**: Код, работающий с общими ресурсами, требующий исключительного выполнения.

Гонка данных

Гонка данных происходит, когда две или более горутины одновременно обращаются к одной и той же переменной, и хотя бы одна из них выполняет запись. Это может привести к непредсказуемым результатам, так как порядок операций не гарантирован. Гонки данных являются подмножеством состояния гонки и выявляются инструментами, такими как `-race` в Go.

Пример кода

```
package main

import (
    "fmt"
    "sync"
)

func main() {
    var counter int
    var wg sync.WaitGroup
    wg.Add(2)

    go func() {
        for i := 0; i < 1000; i++ {
            counter++  // Критическая секция без защиты — возникает гонка данных
        }
        wg.Done()
    }()

    go func() {
        for i := 0; i < 1000; i++ {
            counter++  // Критическая секция без защиты — возникает гонка данных
        }
        wg.Done()
    }()

    wg.Wait()
    fmt.Println("Counter:", counter)  // Не всегда 2000 из-за race condition
}
```

Состояние гонки

Состояние гонки — это более широкий класс ошибок, возникающих из-за отсутствия синхронизации при конкурентном доступе к общим ресурсам. Оно может включать гонки данных, а также другие проблемы, такие как неправильный порядок выполнения операций или некорректное обновление состояния. Например, состояние гонки может возникнуть при доступе к сложным структурам данных без защиты.

Пример состояния гонки при работе с картой

```
package main

import (
  "fmt"
  "sync"
)

func main() {
    m := make(map[int]int)
    var wg sync.WaitGroup wg.Add(2)

    go func() {
        for i := 0; i < 100; i++ { m[i] = i // Запись в карту
        }
        wg.Done()
    }()

    go func() {
        for i := 0; i < 100; i++ {
            m[i] = i + 100 // Запись в карту
        }
        wg.Done()
    }()

    wg.Wait()
    fmt.Println("Map size:", len(m)) // Результат непредсказуем
}
```

Проблема: Карта (`map`) в Go не является потокобезопасной, и конкурентная запись вызывает состояние гонки, которое может привести к панике или некорректным данным. Запуск с `-race` выявит проблему.

Мьютексы

Мьютексы (`sync.Mutex`) — это механизм синхронизации, обеспечивающий эксклюзивный доступ к общим ресурсам. Мьютекс имеет два состояния: заблокирован (`Lock`) и разблокирован (`Unlock`). Только одна горутина может захватить мьютекс, остальные блокируются до его освобождения. Это предотвращает гонки данных, защищая критические секции.

Как мьютексы решают проблему

Мьютексы гарантируют, что только одна горутина выполняет критическую секцию в момент времени, предотвращая одновременный доступ к общей переменной или ресурсу. Например, в случае с `counter` мьютекс защищает операцию инкремента.

Исправление гонки данных с помощью мьютекса:

```
package main

import (
  "fmt"
  "sync"
)

func main() {
   var counter int
   var mu sync.Mutex
   var wg sync.WaitGroup wg.Add(2)

   go func() {
      for i := 0; i < 1000; i++ {
         mu.Lock()
         counter++ // Критическая секция
         mu.Unlock()
      }
      wg.Done()
   }()

   go func() {
      for i := 0; i < 1000; i++ {
         /**
         Мьютекс (`mu`) защищает доступ к `counter`, обеспечивая атомарность операций. Запуск с `-race` не
         выявит проблем.
         */ mu.Lock()
         counter++ // Критическая секция
         mu.Unlock()
      }
      wg.Done()
   }()

   wg.Wait()
   fmt.Println("Counter:", counter) // Всегда 2000
}
```

Критическая секция

Критическая секция — это часть кода, которая обращается к общим ресурсам (например, переменным, файлам, структурам данных) и должна выполняться только одной горутиной в момент времени, чтобы избежать гонок данных или состояния гонки. В Go критические секции обычно защищаются мьютексами, каналами или другими примитивами синхронизации.

Критическая секция при обновлении структуры:

```
package main

import (
  "fmt"
  "sync"
)

type BankAccount struct {
  balance int
   mu sync.Mutex
}

func (a *BankAccount) Deposit(amount int) {
   a.mu.Lock()
   a.balance += amount // Критическая секция a.mu.Unlock()
}

func (a *BankAccount) Withdraw(amount int) bool {
   a.mu.Lock()
   if a.balance >= amount {
      a.balance -= amount // Критическая секция
      a.mu.Unlock()
      return true
   }
   a.mu.Unlock()
   return false
}
```

```
func main() {
    account := &BankAccount{balance: 100}
    var wg sync.WaitGroup
    wg.Add(2)

 go func() {
   for i := 0; i < 100; i++ {
   account.Deposit(1)
  }
   wg.Done()
 }()

  go func() {
    for i := 0; i < 50; i++ { account.Withdraw(2)
   }
   wg.Done()
  }()

  wg.Wait()
  fmt.Println("Final balance:", account.balance)
}
```

Методы `Deposit` и `Withdraw` используют мьютекс для защиты критической секции, где обновляется `balance`. Это предотвращает гонки данных.

Описание

Неправильное использование мьютексов может привести к новым проблемам, таким как взаимные блокировки (deadlock), забытые разблокировки или избыточная блокировка, снижающая производительность. Ниже приведены два примера ошибок.

Взаимная блокировка (Deadlock)

```
package main

import (
   "fmt"
   "sync"
)

type Resource struct {
  mu sync.Mutex
}

func main() {
   r1, r2 := &Resource{}, &Resource{}
   var wg sync.WaitGroup wg.Add(2)

   go func() {
      r1.mu.Lock()
      fmt.Println("Горутина 1 захватила r1")
      r2.mu.Lock() // Ожидает r2 fmt.Println("Горутина 1 захватила r2")
      r2.mu.Unlock()
      r1.mu.Unlock() wg.Done()
   }()

   go func() {
      r2.mu.Lock()
      fmt.Println("Горутина 2 захватила r2")
      r1.mu.Lock() // Ожидает r1 fmt.Println("Горутина 2 захватила r1")
      r1.mu.Unlock()
      r2.mu.Unlock() wg.Done()
   }()

   wg.Wait()
}
```

Проблема: Горутина 1 захватывает `r1` и ждет `r2`, а горутина 2 захватывает `r2` и ждет `r1`. Это вызывает взаимную блокировку, и программа зависает. Запуск с `-race` не выявит проблему, так как это не гонка данных, а логическая ошибка.

Решение: Использовать фиксированный порядок захвата мьютексов или `sync.RWMutex` для разделения чтения и записи.

Забытая разблокировка

```
package main

import (
  "fmt"
  "sync"
)

func main() {
    var mu sync.Mutex
    var wg sync.WaitGroup wg.Add(2)

go func() {
     mu.Lock()
     fmt.Println("Горутина 1 работает")
        // Забыли mu.Unlock() wg.Done()
    }()

    go func() {
        mu.Lock() // Блокируется навсегда
        fmt.Println("Горутина 2 работает")
        mu.Unlock()
        wg.Done()
    }()

    wg.Wait()
}
```

Первая горутина забывает вызвать `Unlock` , из-за чего вторая горутина блокируется навсегда. Программа зависает.

Решение: Использовать `defer mu.Unlock()` сразу после `Lock`:

```
mu.Lock()
defer mu.Unlock()
```

- **Производительность**: Мьютексы добавляют накладные расходы, поэтому их следует использовать только для защиты критических секций. Для интенсивного чтения лучше применять `sync.RWMutex`.

- **Альтернативы**: Каналы могут заменить мьютексы в некоторых сценариях, обеспечивая коммуникацию вместо общей памяти.

Цитирование

- Go Blog: Race Detector
- Go 101: Concurrency Synchronization
- Golangbot: Mutexes
- Go by Example: Mutexes